Algorithmic Deltas and Incremental Maintenance: Encapsulating Real-Time Analytics in PostgreSQL

Zachary Frappier
Department of Computer Science
California State University, Northridge
Email: zachary.frappier.132@my.csun.edu

***Abstract*—Normalized relational databases protect integrity but joins and aggregations can be expensive. One solution is a PostgreSQL trigger-maintained physical summary table. Using the AeroLogistics dataset containing 500,000 deliveries and 500,000 route logs, two comparisons were made. First comparison was for reads, using "on-the-fly" aggregation versus summary-table reads. The second comparison was for writes, using 10,000 row inserts and measuring the time with a trigger against the time without a trigger. Results found were that read latency decreased from an average of 171.594 ms to 0.015 ms using the summary table, showing that using summary reads is approximately 11,440x faster. Using a trigger for writes increased write times from 196.915 ms to 625.979 ms, showing that trigger maintenance introduced 217.89% overhead. These results indicate that incremental view maintenance (IVM) is useful for read-heavy real-time analytics when the additional write cost is acceptable.**

I. Introduction

A. Business and Technical Problem

Real-time operational dashboards often repeatedly execute joins and aggregations over normalized transactional data. Although normalization protects consistency and limits redundancy, repeated analytical recomputation introduces increased read latency as operational datasets grow. In the Aerologistics example deliveries must be monitored, delays greater than 60 minutes violate the service level agreement (SLA), and each minute over 60 produces a $10 penalty.

B. Theoretical Foundation

In Codd's relational model, normalized relations promote logical independence and reduce redundancy [1]. However normalized data requires joins, introducing a competing performance concern. Blakeley et al.'s delta-based materialized view maintenance addresses this by only applying the effects of the changed rows instead of rerunning the complete query [2].

C. Research Objective

The question remains to what extent can trigger based IVM reduce analytical read latency, and what write overhead does this optimization introduce? This simulation aims to show that a summary will make read times faster, and that trigger enabled inserts will be slower because each qualifying row generates additional lookups, calculations and updates.

D. Contributions

This paper's contributions include implementing an on-the-fly SLA aggregation view and a trigger maintained summary, measuring read latency across three executions and the cost of inserting 10,000 records with and without maintenance, and finally evaluating read/write trade-off using empirical results and prior literature.

II. Background

A. Relational Model and Normalization

Codd's relational model organizes data into logically related relations using primary and foreign keys to preserve relationships without repeating descriptive data [1]. In this schema, Deliveries references Hubs and Customers while Route_Logs references Deliveries. Retrieving information that exists across tables such as region, customer priority, and delay, requires joins. Normalization improves integrity but can increase analytical query cost.

B. Baseline SLA Query

Script `01_balinesquery.sql`'s view `vw_sla_on_the_fly` joins `Route_Logs`, `Deliveries`, `Hubs` and customers. Then filters where `Delay_Minutes` > 60, grouping by `Hub_Region` and `Customer_Priority_Level`, and calculating Penalty = (`Delay_minutes` - 60) x $10. Every query against the view repeats the joins, filter, grouping, count, and penalty calculation.

C. Physical Summary Structure

Script `02_create_summary_table.sql`'s trigger-maintained physical summary table `Fact_SLA_Region_Summary` stores precomputed results. The tables composite key is `Hub_Region` and `Priority_Level`, while its measures are `Total_Violations` and `Total_Penalty_Cost`.

D. Incremental View Maintenance

Blakeley et al. describe maintaining derived results using deltas, or changes, rather than complete recomputation [2]. For this implementation, a delta is the contribution of one inserted or updated route log. Script `03_create_ivm_trigger.sql` performs two operations, subtracting old contributions during an update, and

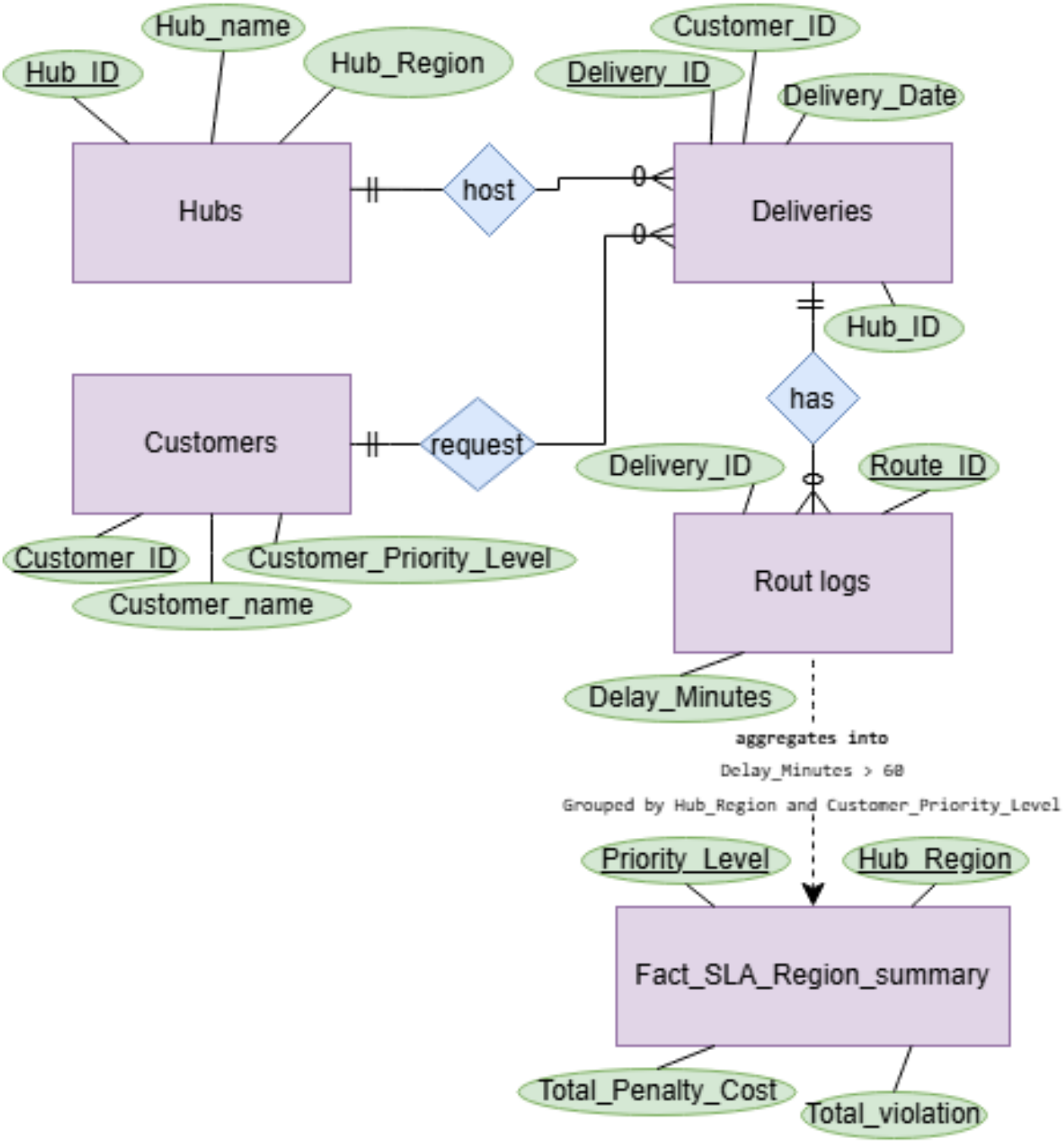


Fig. 1. Logical schema and data lineage for the SLA summary. Route-log records obtain hub region and customer priority through their associated deliveries before being aggregated into the trigger-maintained summary table.

adding or upserting new contributions when the new delay exceeds 60. This approach follows the modern IVM implementation for real-time dataflow architecture [3], [4].

III. Methodology

A. Experimental Environment

The database and benchmark scripts were executed locally on a Windows 11 system equipped with 32 GB of RAM and an eight-core AMD Ryzen 7800 processor operating at 4.20 GHz. Visual Studio Code was used to manage the project files, while

TABLE I
LOCAL EXPERIMENTAL ENVIRONMENT

PostgreSQL version:
pgAdmin version:
OS:
Processor and RAM:

pgAdmin was used to execute the PostgreSQL queries and inspect the benchmark results.

B. Dataset

5 hubs, 500 drivers, 50,000 customers, 500,000 deliveries, 600,000 packages, 500,000 initial route logs, with randomized route distances, delays, assignments, and dates. Only Hubs, Customers, Deliveries, and Route_Logs were required by SLA calculation.

C. Implementation Process

- Script 01: creates the full on-the-fly aggregation view
- Script 02: creates the empty physical summary table
- Script 03: defines the attaches the row-level insert/update trigger.
- Script 04: performs the read-latency comparison
- Script 05: inserts 10,000 rows with the trigger disabled
- Script 06: rebuilds the summary and inserts 10,000 rows with the trigger enabled.

The SQL scripts, benchmark results, and figure-generation files used in this experiment are available in the accompanying GitHub repository.[1]

D. Read-Latency Experiment

One warmup run performed for each query, three measured runs each for `vw_sla_on_the_fly` and `Fact_SLA_Region_Summary` were averaged. The independent variable was the query strategy, while the dependent variable averaged PostgreSQL execution time in milliseconds.

E. Write Penalty Experiment

Script 05 disables the trigger, then inserts 10,000 route logs. Script 06 enables the trigger and inserts another 10,000 route logs. Both tests used equivalent generated values and valid keys, using three runs to find the average of the resulting execution time. The independent variable was the trigger, while the dependent variable was average insertion execution time.

F. Correctness Controls

Baseline and summary results were ordered by region and priority, and compared with the baseline view. Trigger enabled inserts changed summary counts and penalties, the trigger disabled test did not update the summary, and the summary was rebuilt after unsupported deletion operations

[1]Project source-code repository.

IV. RESULTS

The trigger-maintained summary produced lower read latency than the on-the-fly view, while the trigger-enabled insertion required more execution time than insertion without summary maintenance.

A. Read Latency

On-the-fly average was 171.594 ms while summary average was 0.015 ms. Read speedup was calculated as on-the-fly average over 0.015, showing that summary was 11,439.6 times faster. This difference exists because the baseline view joins and aggregates many records while the summary query only scans already grouped views. Latency reduction was the difference of on-the-fly minus summary over on-the-fly multiplied by 100 which was 99.991%.

B. Write Penalty

With trigger disabled 10,000 writes took 196.915 ms while trigger enabled took 625.979 ms. Relative execution time, trigger enabled over trigger disabled, results in showing that trigger enabled took 3.18 times longer to write. Additional time can be calculated by taking the difference of the two, which was 429.064 ms. Trigger overhead was calculated by taking additional time over trigger disabled multiplied by 100 which resulted in 217.89%, which is trigger maintenance cost.

V. DISCUSSION

A. Interpretation of the Result

Original hypothesis that utilizing triggers during writing creates overhead, and that utilizing summary tables for read increases read speedup time was supported. Precomputation shifted aggregation work from query time to update time, resulting in faster reads because no joins or grouping were required. Writes become slower because every violation initiates the trigger process.

B. Connection to Prior Theory

Codd explains that normalized relations reduce redundancy and preserve logical organization [1]. The baseline view demonstrates the need to reconstruct related information through joins. Blakely et al.'s delta-maintenance concept explains updating only changed contributions avoids full recomputation [2]. In this experiment, the trigger operationalizes incremental computation by applying each route log's delta to the existing SLA summary instead of repeatedly joining and aggregating the complete operational dataset.

C. Architectural Recommendation

A hybrid design could use these findings by retaining normalized tables as authoritative transaction source, using incrementally maintained summaries for read-heavy dashboards, performing periodic full reconciliation against the baseline view, and using batch or statement level maintenance if row-level triggers become expensive. When triggers cannot support the write rate, change data capture or streaming maintenance can be considered [4]. Additionally monitoring summary

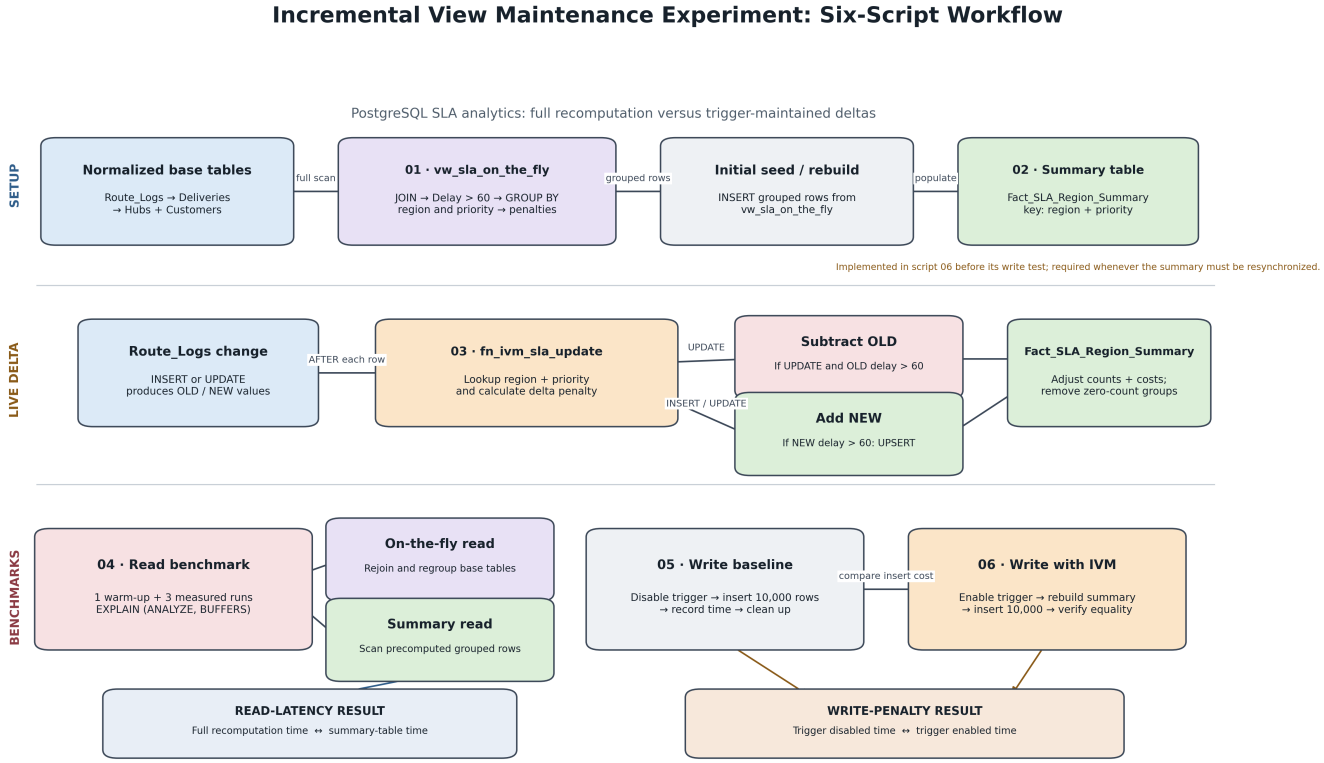


Fig. 2. Experimental workflow for creating the baseline aggregation, applying row-level deltas, and benchmarking read latency and write overhead.

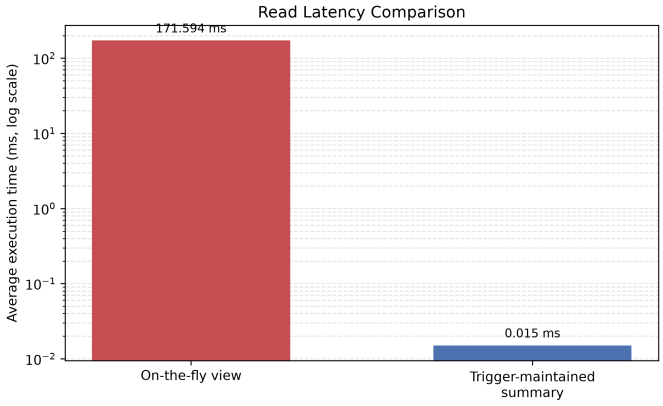


Fig. 3. Average execution time for the on-the-fly aggregation and trigger-maintained summary across three measured runs. The y-axis uses a logarithmic scale.

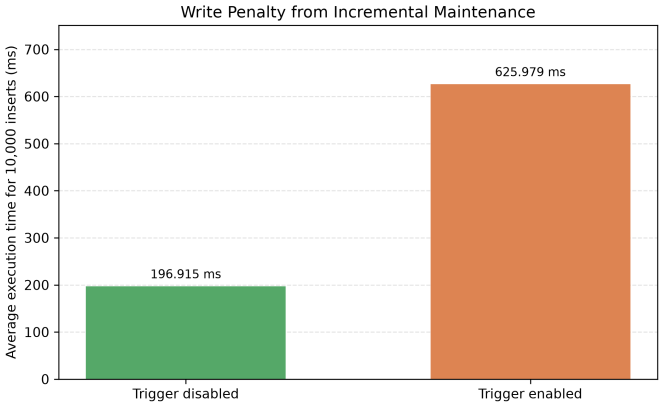


Fig. 4. Average execution time for inserting 10,000 route records with incremental maintenance disabled and enabled.

freshness, trigger failures, and reconciliation differences can help keep a database healthy.

D. Limitation

This experiment was limited to my personal computer's capabilities, in terms of the size of testing. Only three measures were averaged, the summary contained a small number of region priority groups, the generated workload did not model concurrent writers, delays were randomly generated rather than collected from production, and the trigger did not handle deletes. The results of this experiment should not be generalized.

VI. Conclusion

The experiment demonstrated that incremental view maintenance can substantially reduce analytical read latency by transferring aggregation work to the write path. The trigger-maintained summary reduced average read latency from 171.594 ms to 0.015 ms, but increased the execution time for 10,000 inserts from 196.915 ms to 625.979 ms. These findings support the use of maintained summaries for read-intensive

real-time dashboards, provided that the system can tolerate additional write overhead and includes periodic reconciliation to protect correctness.

REFERENCES

[1] E. F. Codd, "A relational model of data for large shared data banks," *Communications of the ACM*, vol. 13, no. 6, pp. 377–387, Jun. 1970, doi: 10.1145/362384.362685.

[2] J. A. Blakeley, P.-Å. Larson, and F. W. Tompa, "Efficiently updating materialized views," in *Proc. 1986 ACM SIGMOD Int. Conf. Management of Data*, Washington, DC, USA, 1986, pp. 61–71, doi: 10.1145/16894.16861.

[3] C. Svingos, A. Hernich, H. Gildhoff, Y. Papakonstantinou, and Y. Ioannidis, "Foreign keys open the door for faster incremental view maintenance," *Proceedings of the ACM on Management of Data*, vol. 1, no. 1, Art. no. 40, May 2023, doi: 10.1145/3588720.

[4] F. McSherry, A. Lattuada, M. Schwarzkopf, and T. Roscoe, "Shared arrangements: Practical inter-query sharing for streaming dataflows," *Proceedings of the VLDB Endowment*, vol. 13, no. 10, pp. 1793–1806, 2020, doi: 10.14778/3401960.3401974.